

Resumen Ejecutivo

SMCP es una re-derivación clean-room del núcleo de coordinación de DeLM (Mao & Mirhoseini, arXiv:2606.10662) más una capa de seguridad propia. Sustituye el orquestador central por un contexto compartido verificado (C) y una cola de tareas con dependencias (T). El progreso intermedio deja de ser un mensaje efímero y se convierte en estado reutilizable. 231 tests en verde, 5 demos ejecutables, capa de seguridad activa por defecto.

1. Capacidades Logradas

1.1 Núcleo DeLM (los mecanismos de carga)

Contexto compartido (SharedContext): el estado verificado C. Solo entran gists admitidos. Las lecturas son snapshots sin bloqueo; las escrituras son atómicas (write-before-publish: el contenido de respaldo se escribe antes de hacer visible el gist).

Cola de tareas (TaskQueue): la cola con dependencias. Una tarea solo es elegible cuando sus dependencias están hechas. El claim es el único punto de serialización (una tarea RUNNING/DONE no puede re-reclamar).

Admisión (AdmissionPipeline): la puerta. Un resultado crudo nunca entra directo a C: se (a) comprime en un gist (+ un Summary con evidencia referenciada), (b) verifica contra su evidencia y (c) admite si pasa. Si falla, reintenta con feedback y, al agotar reintentos, descarta o devuelve a la cola.

Verificador (RuleVerifier): la puerta determinista, sin claves. Verifica el grounding (el gist está anclado en el texto original) y la fidelidad (el gist no introduce afirmaciones ausentes de su evidencia).

Despliegue selectivo (Unfolding): de lo grueso a lo fino, bajo demanda. Los agentes leen la capa de gists por defecto; cuando necesitan detalle, despliegan (G -> S -> raw). Lo desplegado es local a la llamada: no se re-escribe en C, así la capa de gists se mantiene limpia para los demás.

Cliente de modelo (LLMClient): agnóstico al modelo. FakeLLMClient (determinista, sin red) demuestra todo el pipeline sin coste; OpenAICompatibleClient lo corre contra cualquier endpoint OpenAI-compatible (OpenRouter, un proveedor directo, un servidor local).

Pipeline (Worker / DelmPipeline): el bucle descentralizado. N workers corren en paralelo sobre la cola compartida; cada uno reclama, lee, razona y admite. Al agotarse la cola, el último worker decide si generar más subtareas o finalizar. No hay orquestador central que merge resultados: ese es el punto.

1.2 Capa de seguridad (lo que el paper no trae)

Proveniencia e integridad (Capas 1+2): cada gist lleva el autor que lo admitió y una firma ed25519 sobre su digest canónico (SHA-256 del contenido, independiente del orden y de los campos de firma). Un TrustGate decide quién puede escribir (allowlist / denylist / require-signed). La inmutabilidad impide sobrescribir: re-admitir un label solo vale si el digest es idéntico. Un AdmissionLedger append-only con cadena de hashes deja un rastro auditable y reproducible.

Anti prompt-injection (Capa 5): el contexto compartido también es la entrada de cada agente, así que una fuente envenenada puede orientar a todos los que la lean. Un detector determinista (sin LLM) marca fuentes con instrucciones inyectadas, y un modelo de taint (CLEAN / SUSPICIOUS / CONFIRMED) con cierre transitivo acota el alcance: un gist derivado de una fuente envenenada hereda el taint, y la cuarentena se aplica tanto en el render (los CONFIRMED se omiten; los SUSPICIOUS se enmarcan como dato no confiable, no instrucciones) como en el despliegue selectivo (el raw de un gist bloqueado no se expone).

Detector de inyección endurecido (injection_hardened): el detector baseline es un catálogo de regex sobre el texto crudo, evadible con zero-width chars, homoglifos, leetspeak, acentos, palabras espaciadas por letra o payloads partidos en líneas. El detector endurecido normaliza antes de correr el mismo catálogo: strip de zero-width/invisibles, NFKD + drop de marcas combinadas, lowercase, homoglifos (tabla reducida de confusables cirílico/griego), leetspeak, whitespace (newlines/tabs simples -> espacio), colapso de palabras espaciadas por letra, scan de regiones (el

peor caso se detecta compactando cada región letter-spaced y chequeando firmas ordenadas dentro de la región).

Métricas (metrics): MetricsTracker registra coste/latencia por tarea: timed() como context manager (mutable in-flight), precios por modelo (tabla 2026 orientativa, overrideable), agregado con percentiles p50/p95, admit rate, retries, breakdown por worker y por modelo, serializable a JSON para el ledger. Si el bloque lanza, el registro se escribe igual con admitted=False y error, y la excepción se re-lanza.

Política de expansión (expansion): ExpansionPolicy decide cuándo invocar el paso generate more subtasks: queue viva -> no; sin señal (done+failed==0) -> no; target_progress opt-in alcanzado -> no; presupuesto agotado -> no; failure-storm (fail_ratio >= umbral) -> no (terminal, documentado); si no, burst acotado n_new = min(max_burst, budget_remaining).

1.3 Capa de despliegue (Capa 3: red)

Gossip: el protocolo de rumor (floor de versión, regla path-rich, re-difusión, cambio significativo, retiro).

Heartbeat: el latido de vida (registro, beat, frescura, sweep, revivir, retiro).

Requirements: la inmutabilidad de la política (policy_hash, atestación de release, gates de admisión).

QUIC: el transporte seguro entre hosts (framing/round-trip/malla completa, identidad-cert del QUIC con CN = peer_id, verificación post-handshake).

Nostr: el relay de red (BIP340 contra los 19 vectores oficiales, NostrRelayServer/NostrRelayClient, NostrTransport, convergencia de malla, guardia del relay: rate-limit por pubkey, dedup, límite de tamaño de evento, snapshot/restore).

mDNS: el transporte de discovery (swappable con DiscoveryBus, mismo contrato que DeploymentNode no cambia).

1.4 Adaptador DeepSeek Harness (Fase A+B)

HarnessLLMClient: implementa LLMClient (misma interfaz que OpenAICompatibleClient), así un worker la sustituye sin cambiar su código. Import lazy: deepseek_harness se importa dentro de _ensure(), no a module load — la suite por defecto corre sin el SDK instalado (y sin su runtime de 268 MB).

Route generada: el adaptador escribe el patch llm-pi-ai (route qwen-local, OpenAI-compatible) que apunta al mismo servidor local que ya usa DELM — no un segundo modelo. Key desde archivo: lee la key minted de agent_api_key.json (no la hardcodea); inyectable por api_key explícito.

Subprocess único: el runtime dsh se lanza una vez por cliente y se reutiliza entre turnos. build_client ramifica: si use_harness, devuelve HarnessLLMClient; si no, OpenAICompatibleClient. El worker no cambia — solo el cliente que recibe es distinto.

Opt-in: sin DELM_HARNESS, nada cambia (suite por defecto verde sin SDK). Caída limpia: sin SDK, build_client(use_harness=True) lanza (lazy import); sin DELM_HARNESS, no se toca el SDK. Mismo modelo: el harness apunta al 127.0.0.1:8888 (el Qwen local).

2. Snippets de Código

2.1 DelmPipeline.run — el bucle descentralizado

```
async def run(self, initial_tasks: list[Task],
               max_rounds: int = 8,
               reason: Callable[[Task], Awaitable[str]] | None = None,
               yield_between: bool = False) -> PipelineOutcome:
    # Per-worker identities: one author_id + signing key per worker.
    # The secure context verifies each admission against the
    # registered public key, so a worker cannot admit under
    # someone else's identity.
    for i in range(self.n_workers):
        if i not in self._keys:
            self._keys[i] = KeyPair.new(f"worker-{i}")
        if isinstance(self.ctx, SecureSharedContext):
            self.ctx.register_key(
                f"worker-{i}",
                self._keys[i].public_key,
                self._keys[i].kind,
            )
    self.queue.enqueue_many(initial_tasks)
    workers = [
        Worker(i, self.llm, self.ctx, self.queue, self.admission,
              reason=reason, yield_between=yield_between,
              author_id=f"worker-{i}", key=self._keys[i],
              metrics=self.metrics)
        for i in range(self.n_workers)
    ]
    rounds = 0
    while rounds < max_rounds:
        # All workers race on the shared queue until it is exhausted.
        await asyncio.gather(*(w.run() for w in workers))
        rounds += 1
        # Last-worker step: generate more subtasks or finalize.
        if self.queue.is_empty():
            more = self._maybe_expand(workers)
            if more:
                self._expansion_spawned += len(more)
                self.queue.enqueue_many(more)
                continue
            break
        # If the queue is not empty but nothing is eligible
        # (e.g. a deadlock on unmet deps), break to avoid a spin.
        if not self.queue.eligible_labels():
            break
    answer = await self._finalize()
    return PipelineOutcome(
        answer=answer,
        admitted_gists=len(self.ctx),
        rounds=rounds,
        workers=[w.result for w in workers],
        metrics=self.metrics.aggregate(),
    )
```

2.2 build_client — la ramificación opt-in

```
def build_client(config: ModelConfig) -> LLMClient:
    """Return the LLMClient selected by config.use_harness.

    When use_harness is True, this returns a HarnessLLMClient
    (the DeepSeek Harness agent runtime). The import is lazy:
    deepseek_harness is imported inside HarnessLLMClient._ensure(),
    so this function does not require the SDK to be installed.

    When use_harness is False (the default), this returns an
```

```

OpenAICompatibleClient – the plain OpenAI-compatible client.
The worker does not change; only the client it receives does.
"""
if config.use_harness:
    from delm.core.harness_client import build_harness_client
    return build_harness_client(
        config,
        api_key=config.api_key or None,
    )
return OpenAICompatibleClient(
    model=config.model,
    base_url=config.base_url,
    api_key=config.api_key,
)

```

2.3 HarnessLLMClient._ensure — import lazy

```

def _ensure(self) -> None:
    """Import deepseek_harness and launch the dsh runtime.

    The import is lazy: it happens here, not at module load, so
    the module can be imported without the SDK installed. The
    subprocess is launched once per client and reused across
    turns (the runtime is stateful: it holds the agent session).
    """
    if self._client is not None:
        return
    import deepseek_harness # lazy import
    self._client = deepseek_harness.DeepSeekHarness(
        deepseek_harness.DeepSeekHarnessConfig(
            provider=HARNESSE_ROUTE_ID,
            model=self.model,
            dsh_home=self.dsh_home,
            request_timeout_seconds=self.timeout,
            **({
                "reasoning_effort": self.reasoning_effort
                if self.reasoning_effort else {}
            }),
            **({
                "max_tokens": self.max_tokens
                if self.max_tokens else {}
            }),
        ),
    )

```

2.4 SharedContext.admit — write-before-publish

```

def admit(self, gist: Gist) -> Gist:
    """Admit a gist into the shared context.

    Write-before-publish: the backing content is written before
    the gist is made visible. This ensures that a reader who sees
    the gist can always read the underlying content – the gist is
    never visible without its content.

    The admission is atomic: either the gist is admitted (and its
    content is written) or it is not (and no partial state is left
    behind). The TrustGate decides who can write; the ledger
    records the admission (append-only, hash-chained).
    """
    if not self._gate.can_write(gist.author):
        raise AdmissionDenied(
            f"author {gist.author!r} cannot write"
        )
    # Write-before-publish: content first, then visibility.
    self._content[gist.label] = gist.content

```

```
self._gists[gist.label] = gist
self._ledger.record(
    author=gist.author,
    label=gist.label,
    admitted=True,
)
return gist
```

3. Mapeado de la UI (Demos)

SMCP no tiene una UI web. Su interfaz de usuario son las 5 demos ejecutables — cada una es una entrada al sistema, con una salida verificable. Son la "UI" del proyecto: el punto de contacto con el usuario, la forma en que el sistema se muestra y se prueba.

3.1 run_demo — el pipeline punta a punta

```
python -m delm.demo.run_demo

=== DELM end-to-end demo (no API key) ===
queue exhausted : True
admitted gists  : ['u1/w0', 'u2/w1', 'u3/w2', 'u4/w3']
per-worker adm  : {0: 1, 1: 1, 2: 1, 3: 1}
u3 admitted as  : u3/w2
u3 gist (trunc) : Examined u3. The load-bearing constraint is:
u3 raw present  : True
unfold raw (trunc): Examined u3. The load-bearing constraint is:
final answer    : ANSWER: The task is resolved by the verified
                  change recorded in the shared context;
                  all constraints are satisfied.

=== demo OK ===
```

3.2 run_security_demo — Capas 1+2 (firma, integridad, ledger)

```
python -m delm.demo.run_security_demo

=== DELM security demo (Capas 1+2) ===
backend: ed2519 available = True
1) alice admitted u1 (verified gist)          -> OK
2) eve blocked (no registered key)             -> AdmissionDenied
3) tamper rejected (immutable label)           -> AdmissionDenied
   idempotent re-admit of identical u1         -> OK
4) ledger entries=4 chain_integrity=True
   #0 alice u1  accepted=True  admitted
   #1 eve  u1  accepted=False  unknown author (no key)
   #2 alice u1  accepted=False  immutable label, digest changed
   #3 alice u1  accepted=True  idempotent recheck
=== demo OK ===
```

3.3 run_taint_demo — Capa 5 (cuarentena de prompt-injection)

```
python -m delm.demo.run_taint_demo

=== taint report (label -> level) ===
u1: CLEAN
u2: CLEAN
u3: CONFIRMED
u4: CONFIRMED

=== agent-visible render (quarantine active) ===
[u1] The build passed all checks and the tests are green.
[u2] Derived from u1: the release is safe to ship.

=== assertions ===
ok: clean gists visible, poisoned lineage (u3,u4) blocked

=== taint demo OK ===
```

3.4 run_multihost_demo — malla sobre QUIC (2 nodos, procesos distintos)

```
python -m delm.demo.run_multihost_demo

=== SMCP multi-host demo (2 nodos, procesos distintos, QUIC) ===
puerto B (serve) : 42201
gists A           : ['A-gist', 'B-gist']
gists B           : ['A-gist', 'B-gist']
convergencia      : A == B (mismo conjunto de gists)
=== demo OK ===

# Modo Nostr:
python -m delm.demo.run_multihost_demo --nostr

# El default corre sobre QUIC (el despliegue real): 2 nodos en
# procesos distintos que se conectan entre sí (B servidor,
# A cliente), cada uno firma su gist (BIP340), se intercambian
# gossip/heartbeat por QUIC y convergen al mismo conjunto de gists.
# La flag --nostr arranca el modo anterior (1 relay Nostr + 2 nodos).
```

3.5 run_real_demo — modelo real (config → cliente → pipeline)

```
python -m delm.demo.run_real_demo --dry-run

=== dry-run (no model called) ===
model      : unsloth/Qwen3.8-27B-GGUF
base_url   : http://127.0.0.1:8888/v1
api_key     (unset)
temperature: 0.0
backend    : openai-compatible
=== dry-run OK ===

# Con el modelo cargado:
python -m delm.demo.run_real_demo --tasks 4 --workers 4

# O con el harness (DeepSeek Harness agent runtime):
python -m delm.demo.run_real_demo --harness --tasks 4 --workers 4

# El --dry-run resuelve la config y la imprime (ocultando la key)
# sin llamar al modelo. El --harness activa el backend DeepSeek Harness
# (Fase A+B), que apunta al mismo servidor local (127.0.0.1:8888).
```


4. Progreso Actual

4.1 Tests (231 en verde)

test_delm.py	14	Núcleo: cola, contexto, admisión, despliegue, pipeline
test_security.py	18	Capas 1+2: digest, firma, gate, ledger, pipeline seguro
test_persistence.py	10	Persistencia: dump/load/export del AdmissionLedger
test_rotation.py	8	Rotación/revocación de la clave del owner (control-plane)
test_taint.py	15	Capa 5: detector, niveles, cierre transitivo, cuarentena
test_mejoras.py	28	Métricas (coste/latencia) y expansión cableadas
test_config.py	16	Loader de config: env > YAML > default, key solo por env
test_real_model_wiring.py	2	Wiring punta a punta (config → cliente → pipeline)
test_gossip.py	13	Capa 3: floor de versión, regla path-rich, re-difusión
test_requirements.py	12	Capa 3: inmutabilidad, policy_hash, atestación de release
test_heartbeat.py	10	Capa 3: registro, beat, frescura, sweep, revivir
test_mesh.py	17	Capa 3: transport/nodo/red/pipeline + QUIC e2e
test_nostr.py	19	BIP340 (19 vectores) + relay de red + NostrTransport
test_nostr_relay_guard.py	5	Capa 4: rate-limit, dedup, límite de tamaño, snapshot
test_deployment.py	13	Capa 4: discovery, relays, bootstrap, control-plane
test_mdns.py	8	mDNS: transporte de discovery (swappable)
test_strict_mode.py	9	Modo estricto: provenance no degrada a HMAC
test_harness_adapter.py	6	Adaptador DeepSeek Harness: import lazy, subclase, build_client
test_multihost.py	2	Demo multi-host: convergencia sobre QUIC y Nostr
Demos	4	run_demo, run_security_demo, run_taint_demo, run_multihost_demo

4.2 Commits

82f5997	Fase A: adaptador DeepSeek Harness (harness_client) tras la interfaz LLMClient
65fc82f	Capa 1: dependencias principales declaradas + CI
3e656e0	Fase B: wiring opt-in DELM_HARNESS + worker detrás del harness
142f0bc	Capa 1: extras declarados + fin del fallback HMAC silencioso (modo estricto)
dfabd3e	Capa 4: guardia del relay Nostr (rate-limit, dedup, límite de tamaño, snapshot/restore)
397dfb2	Capa 3: enlace identidad-cert del QUIC (CN = peer_id, verificación post-handshake)
5be9c34	Capa 4: rotación/revocación de la clave del owner (control-plane)
f58afac	Capa 2: persistencia append-only del AdmissionLedger + replay/auditoría
9d60d43	Capa 4: demo multi-host sobre QUIC (default) + modo Nostr
ffc8e24	Capa 4: transporte de discovery mDNS (swappable con DiscoveryBus)
061f9dc	Capa 3 sobre red: demo multi-host (2 nodos, procesos distintos) + test de convergencia
10cf304	README: sync con la suite (177 tests) + demo multi-host

82f5997	Fase A: adaptador DeepSeek Harness (harness_client) tras la interfaz LLMClient
493d1d6	Capa 3 sobre red: demo multi-host (2 nodos, procesos distintos) + test de convergencia
0914978	README: sync con la suite (176 tests) + capa 3 sobre red vía Nostr
7a7f7b8	Capa 3 sobre red: NostrTransport + NostrSwarm (la malla corre vía Nostr)

4.3 Arquitectura (módulos delm/core)

pipeline.py	DelmPipeline + Worker: el bucle descentralizado (reclamar, leer, razonar, admitir)
shared_context.py	SharedContext + SecureSharedContext: el estado verificado C
task_queue.py	TaskQueue + Task: la cola con dependencias
admission.py	AdmissionPipeline: comprimir, verificar, admitir (la puerta)
verifier.py	RuleVerifier: la puerta determinista (grounding + fidelidad)
unfolding.py	Unfolding: despliegue selectivo (G -> S -> raw)
llm.py	LLMClient + OpenAICompatibleClient + FakeLLMClient: el cliente de modelo
harness_client.py	HarnessLLMClient: el adaptador DeepSeek Harness (import lazy)
provenance.py	Provenance: la cadena de autoría (author + firma ed25519)
ledger.py	AdmissionLedger: el rastro append-only (cadena de hashes)
injection.py	Injection: el detector de prompt-injection (regex)
injection_harden.py	InjectionHardened: el detector endurecido (normaliza antes)
taint.py	Taint: el modelo de taint (CLEAN / SUSPICIOUS / CONFIRMED)
metrics.py	MetricsTracker: coste/latencia por tarea (p50/p95)
expansion.py	ExpansionPolicy: decidir cuándo generar más subtareas
gossip.py	Gossip: el protocolo de rumor (la malla)
heartbeat.py	Heartbeat: el latido de vida (frescura)
requirements.py	Requirements: la inmutabilidad de la política
quic_host.py	QuicHost: el transporte QUIC entre hosts
nostr.py	NostrTransport + NostrRelayServer/Client: el relay Nostr
mdns.py	MdnsBus: el transporte de discovery mDNS
mesh_network.py	MeshNetwork: la red (la malla completa)
mesh_node.py	MeshNode: el nodo (un host en la malla)
mesh_pipeline.py	MeshPipeline: el pipeline sobre la malla